Bank Loan SQL Analysis — Cell-by-Cell / Query-by-Query Guide

Purpose

This SQL analysis is designed around the Bank Loan Portfolio & Credit Risk dashboard problem statement.

The existing project SQL contains the core business queries plus advanced SQL topics such as CTEs and window functions. This version organizes those analyses into a clean, business-first workflow that can be used for project documentation and interviews.

The project definition is:

- **Good Loan:** `Fully Paid` or `Current`
- **Bad Loan:** `Charged Off`
- `INT_RATE` and `DTI` are stored as decimals, so `0.12` represents `12%`.

Part 1 — Data Understanding & Validation

Query 1 — Check the database and table

```
USE bank_loan_analytics;
DESCRIBE loan_data;
SELECT COUNT(*) AS total_rows FROM loan_data;
```

What are we doing?

We first make sure SQL is connected to the correct database and inspect the table structure.

Why?

Before calculating KPIs, we need to know:

- What columns exist
- How the fields are stored
- How many records are available

Query 2 — Duplicate validation

We check full-row duplicates and duplicate loan IDs.

Why?

`ID` represents the loan application. Duplicate IDs could inflate:

- Total applications
- Funded amount
- Received amount
- Good/Bad Loan counts

The project Python validation found **0 duplicate rows and 0 duplicate loan IDs**, so this SQL validation should be used as a cross-check.

Query 3 — Missing-value validation

We check missing `EMP_TITLE`.

Why?

Missing employment title does not mean the loan application itself is invalid. The project analysis found 1,438 missing employment titles, approximately 3.73% of the dataset, and retained those loan records.

Query 4 — Negative-value validation

We test:

- Negative loan amount
- Negative DTI
- Negative interest rate

Why?

These are business-quality checks. Negative values would indicate invalid portfolio records.

Part 2 — Summary Dashboard KPIs

Query 5 — Total Loan Applications

```
SELECT COUNT(*) AS total_applications
FROM loan_data;
```

Business question

How many loan applications are in the portfolio?

Dashboard use

Summary Page → Total Loan Applications

Query 6 — Total Funded Amount

```
SELECT SUM(LOAN_AMOUNT) AS total_funded
FROM loan_data;
```

Business question

How much money did the bank fund?

Dashboard use

Summary Page → Total Funded Amount

Query 7 — Total Amount Received

```
SELECT SUM(TOTAL_PAYMENT) AS total_received
FROM loan_data;
```

Business question

How much money has been received from borrowers?

Dashboard use

Summary Page → Total Amount Received

Query 8 — Average Interest Rate

```
SELECT ROUND(AVG(INT_RATE) * 100, 2)
FROM loan_data;
```

Why multiply by 100?

The raw field stores interest rate as a decimal.

Example:

`0.12 × 100 = 12%`

Dashboard use

Summary Page → Average Interest Rate

Query 9 — Average DTI

```
SELECT ROUND(AVG(DTI) * 100, 2)
FROM loan_data;
```

Business question

What is the average debt-to-income ratio of borrowers?

Dashboard use

Summary Page → Average DTI

Part 3 — Good Loan vs Bad Loan

Query 10 — Good Loan Applications

Good loans are:

```
Fully Paid
Current
```

We count only those statuses.

Query 11 — Bad Loan Applications

Bad loans are:

```

```

```
Charged Off
```

Query 12 — Good/Bad percentage

We calculate the percentage of all applications classified as Good or Bad.

Why?

The percentage is more useful for risk monitoring than raw counts alone because it normalizes portfolio size.

Query 13 — Good/Bad Loan summary

We combine:

- Applications
- Funded Amount
- Amount Received

for Good and Bad Loans.

Dashboard use

This supports the Good Loan and Bad Loan cards/donuts in the Summary dashboard.

Part 4 — Loan Status Analysis

Query 14 — Applications by Loan Status

We group applications by `LOAN_STATUS`.

Why?

This shows the composition of the portfolio, such as:

- Fully Paid
- Current
- Charged Off

Dashboard use

Loan Status-wise Loan Applications

Query 15 — Loan Status percentage share

We calculate each status as a percentage of total applications.

Why?

This gives a normalized view of portfolio composition.

Query 16 — Funded vs Received by Status

We calculate:

- Number of applications
- Funded amount
- Amount received

for each loan status.

Dashboard use

This supports the Summary page funded-versus-received visual.

Part 5 — Credit Risk Analysis

Query 17 — Bad Loan Rate by Grade

We calculate:

```
Bad Loan Rate =
Charged Off Loans / Total Loans in Grade × 100
```

Why?

Credit grade is a major risk segmentation variable.

The result tells us whether lower-quality grades have higher charge-off rates.

Query 18 — Grades above overall Bad Loan

---

rate

We first calculate the Bad Loan rate for every grade.

Then we calculate the overall portfolio Bad Loan rate.

Finally, we return only grades whose rate is above the portfolio average.

Why?

This identifies grades that require additional credit-risk attention.

Query 19 — Average Interest Rate by Loan Status

We compare interest pricing across loan statuses.

Why?

It helps investigate whether higher-risk outcomes are associated with higher pricing.

Query 20 — Average DTI by Loan Status

We compare borrower DTI across statuses.

Why?

Higher borrower leverage can be an important risk indicator.

Query 21 — Income Groups

We create:

- Low income: `< $40,000`
- Medium income: `$40,000–$74,999`
- High income: `>= $75,000`

Why?

This demonstrates `CASE WHEN` segmentation and provides another borrower-profile perspective.

Part 6 — Time Analysis

Query 22 — Monthly Applications, Funded and Received

We extract:

- Month number
- Month name
- Applications
- Funded amount
- Amount received

Why?

The problem statement requires a monthly loan application trend.

Dashboard use

Overview → Monthly Loan Applications

Query 23 — Latest Available Month KPIs

We identify the maximum `ISSUE_DATE`, then calculate the KPIs for that month.

Why?

This demonstrates a dynamic Month-to-Date style analysis rather than hard-coding a month.

Part 7 — Overview Dashboard

Query 24 — Top States by Applications

We rank states based on application volume.

Dashboard use

Overview → Regional Analysis

Query 25 — State Ranking by Funded Amount

We use:

```
RANK() OVER (...)
```

Why?

This identifies the highest-funded states.

It is also an example of an SQL window function.

Query 26 — Loan Term Analysis

We group by cleaned `TERM`.

Metrics:

- Applications
- Funded Amount

Dashboard use

Overview → Loan Term Analysis

Query 27 — Home Ownership Analysis

We group by:

- RENT
- MORTGAGE
- OWN
- OTHER
- NONE

and calculate applications and funded amount.

Dashboard use

Overview → Home Ownership Analysis

Query 28 — Loan Purpose Breakdown

We group by `PURPOSE`.

Metrics:

- Applications
- Funded Amount
- Amount Received

Dashboard use

Overview → Loan Purpose Breakdown / Treemap

Query 29 — Purpose Percentage Share

We use:

```
SUM(COUNT(*)) OVER ()
```

to calculate the total across all purpose groups.

Why?

This is an example of a window function used to calculate each purpose's contribution to the total portfolio.

Part 8 — Subqueries

Query 30 — Loans Above Average Loan Amount

The inner query calculates:

```
Average Loan Amount
```

The outer query counts loans above that value.

Why?

This demonstrates how a subquery can turn a portfolio-level benchmark into a row-level filter.

Query 31 — States Above Average Application Volume

We first calculate applications by state.

Then we calculate the average number of applications across states.

Finally, we return states above that average.

Why?

This identifies markets with above-average lending activity.

Part 9 — Window Functions

Query 32 — Month-over-Month Funded Amount

We use:

```
LAG()
```

to retrieve the previous month's funded amount.

Then:

```
MoM % =
(Current Month − Previous Month)
/
Previous Month × 100
```

Why?

This measures the rate at which lending activity changes month to month.

Query 33 — Running Total Applications

We use:

```
SUM(applications) OVER (ORDER BY month_no)
```

Why?

This creates cumulative application volume over the year.

Query 34 — Top Purpose Within Each Grade

We use:

```
ROW_NUMBER() OVER (
    PARTITION BY GRADE
    ORDER BY COUNT(*) DESC
)
```

Why?

Instead of finding one overall top purpose, we find the most common purpose **inside every credit grade**.

This is a strong interview example of `PARTITION BY`.

Query 35 — Three-Month Moving Average

We calculate:

```
Current month
+
Previous month
+
Month before previous
```

and average them.

Why?

Moving averages smooth short-term fluctuations and reveal the underlying funding trend.

Part 10 — Power BI Source View

Query 36 — `vw_loan_powerbi`

The SQL view prepares one clean source for Power BI.

It contains:

- Original loan fields
- Trimmed term
- `LOAN_CATEGORY`

The category follows the project definition:

```
Fully Paid / Current → Good Loan
Charged Off → Bad Loan
```

Why use a view?

Power BI can connect to a clean, reusable SQL object instead of rebuilding the business classification every time.

Part 11 — Final Reconciliation

The final query returns all major KPIs together:

- Applications
- Funded Amount
- Amount Received
- Average Interest Rate
- Average DTI
- Good Loan Applications
- Bad Loan Applications

Why?

This is the SQL checkpoint that should reconcile with:

Raw Data → SQL → Python → Power BI

If these numbers do not match, investigate the business definition or transformation before presenting the dashboard.

SQL → Dashboard Mapping

SQL Analysis	Dashboard
Total applications	Summary KPI
Total funded	Summary KPI
Total received	Summary KPI
Average interest rate	Summary KPI
Average DTI	Summary KPI
Good/Bad loan count	Summary
Good/Bad loan percentage	Summary
Loan status applications	Summary
Funded vs received	Summary
Interest rate by status	Summary
DTI by status	Summary
Monthly applications	Overview
State analysis	Overview
Loan term	Overview
Home ownership	Overview
Loan purpose	Overview
Grade risk	Credit-risk analysis

Important correction to the earlier SQL

The original project SQL contains some expressions where **every status other than `Charged Off` is treated as Good Loan**.

For example, its original `CASE` logic uses:

```
CASE
    WHEN LOAN_STATUS = 'Charged Off'
    THEN 'Bad Loan'
    ELSE 'Good Loan'
END
```

But the project definition says:

```
Good Loan = Fully Paid / Current
Bad Loan  = Charged Off
```

Therefore, this analysis deliberately uses:

```
CASE
    WHEN LOAN_STATUS IN ('Fully Paid', 'Current')
        THEN 'Good Loan'
    WHEN LOAN_STATUS = 'Charged Off'
        THEN 'Bad Loan'
    ELSE 'Other'
END
```

This keeps the SQL analysis aligned with the stated business definition rather than silently treating an unexpected future status as Good Loan.

Interview Explanation

A concise explanation of the SQL work is:

> "I started by validating the loan table and checking duplicates, missing values and invalid numeric values. Then I calculated the core portfolio KPIs such as applications, funded amount, received amount, average interest rate and DTI. I classified loans into Good and Bad based on the business definition and analyzed status, grade, purpose, geography, term and home ownership. I then used CTEs, subqueries

and window functions for advanced analysis such as month-over-month growth, running totals, state ranking, top purpose by grade and moving averages. Finally, I created a SQL view so Power BI could consume a consistent business-ready dataset."